

QuickJob

Volledige verbeterlijst & code quality rapport | 5 maart 2026

Code Quality Score

38/100

Totaalscore

25/100

React Patterns

50/100

Backend

20/100

TypeScript

React Component Patterns — 25/100

Monolithische componenten van 1.200+ regels. Geen custom hooks, geen component compositie. `groupJobsByDay()` is gedupliceerd in meerdere bestanden. Geen state management (Context/Redux). `Dashboard.tsx` en `DashboardClient.tsx` moeten elk opgesplitst worden in 5-8 kleinere componenten.

Backend Architectuur — 50/100

Goede route-structuur en RESTful design. Echter: JWT verificatie is een lege stub, geen RBAC middleware, minimale input validatie, geen rate limiting. `Server.js` is netjes maar routes missen beveiliging.

TypeScript Gebruik — 20/100

TypeScript is geconfigureerd met strict mode, maar wordt nauwelijks benut. Excessief gebruik van 'any' types. Slechts 1 type-definitie bestand (`client.ts`). Geen API response types, geen error types, geen generics.

Algehele Structuur — 55/100

Goede scheiding frontend/backend, Supabase integratie werkt, Stripe Connect is operationeel. API service laag (`api.ts`) centraliseert calls netjes. Maar: 779 regels in één bestand, veel duplicatie.

Geschatte opschoontijd

Categorie	Geschatte tijd	Prioriteit
Component refactoring (splitsen dashboards)	8-12 uur	Hoog
Custom hooks + shared utilities	4-6 uur	Hoog
TypeScript types uitbreiden	6-8 uur	Hoog
API service laag refactoren	3-4 uur	Medium
Backend security (JWT + RBAC)	4-6 uur	Kritiek
Error handling & loading states	4-6 uur	Medium
Code duplicatie verwijderen	3-4 uur	Medium
TOTAAL	32-46 uur	—

DEEL 1 — Wat Claude kan doen

Beveiliging (Kritiek)

- 1 JWT Middleware implementeren**
verifyJwt.js is een lege stub — alle routes zijn onbeschermd. Echte JWT-verificatie bouwen met jsonwebtoken.
- 2 Role-Based Access Control**
Geen middleware die controleert of een student bij student-routes komt. Role-checking op alle route-groepen.
- 3 Input validatie versterken**
Veel endpoints doen minimale validatie. Validatielaag toevoegen met zod of express-validator.
- 4 CORS beperken**
CORS staat open voor alles. Beperken tot frontend-origin(s).
- 5 Rate limiting**
Geen bescherming tegen brute-force. express-rate-limit toevoegen op auth-routes.
- 6 File upload beveiliging**
Geen restricties op bestandstype of grootte. Mime-type checking en size limits instellen.

Backend Functionaliteit

- 7 Notificatiesysteem**
Tabel bestaat maar geen endpoints/logica. CRUD-endpoints + triggers bouwen.
- 8 Chat/Messaging**
Tabellen bestaan maar niet gekoppeld. Real-time messaging implementeren.
- 9 Password reset flow**
Geen wachtwoord-reset endpoint. Token-based reset bouwen.
- 10 Email verificatie**
Geen bevestigingsflow na registratie. Verificatie-token systeem opzetten.
- 11 No-show beleid**
Concept staat in businessplan maar niet in code. Penalty/sanctie systeem bouwen.
- 12 Token-systeem (loyaliteit)**
Studenten verdienen tokens per job — nog niet gebouwd. Tokens tabel + logica.
- 13 Contractgeneratie**
Niet geïmplementeerd. PDF-contracten genereren bij job-acceptatie.
- 14 Job lock mechanisme**
Logica ontbreekt. Afdwingen dat geen annulatie na acceptatie mogelijk is.
- 15 Minimumfacturatie 3 uur**
Staat niet in betaallogica. Toevoegen aan payment calculation.
- 16 Admin dashboard stats**
Geen KPIs of metrics. Queries voor omzet, actieve gebruikers, etc.

- 17 Geocoding integratie**
Endpoint bestaat maar doet niets. Koppelen aan OpenStreetMap Nominatim.
- 18 Afstandsfitering**
Haversine berekening ontbreekt. Echte afstandsberekening implementeren.

Frontend & Code Kwaliteit

- 19 Component refactoring**
Dashboard componenten opsplitsen: JobCard, JobsList, FilterPanel, etc. Van 1.200 naar <400 regels per bestand.
- 20 Custom hooks bouwen**
useJobFiltering(), useFetchJobs(), useGeoLocation() — herbruikbare logica extraheren.
- 21 TypeScript types uitbreiden**
Volledige type definitions voor alle entiteiten. 'any' vervangen door echte types.
- 22 Shared utilities**
dateFormatting.ts, jobGrouping.ts, distanceCalculation.ts — DRY maken.
- 23 Error handling verbeteren**
Specifieke, gebruikersvriendelijke foutafhandeling + Error Boundary component.
- 24 Loading states & skeletons**
Skeleton screens en loading spinners toevoegen op alle data-schermen.
- 25 Form validatie frontend**
Realtime veldvalidatie toevoegen op alle formulieren.
- 26 Start-stop uurregistratie**
Timer die 30 min voor shift verschijnt — bouwen in student dashboard.
- 27 API URL configureerbaar**
Hardcoded IP verwijderen, environment variabel maken.
- 28 Error logging (winston/pino)**
Gestructureerde logging toevoegen op backend.
- 29 Database migraties**
Migratie-scripts schrijven zodat schema reproduceerbaar is.
- 30 API documentatie**
Swagger/OpenAPI docs genereren.

DEEL 2 — Wat Denis ZELF moet doen

Accounts & Externe Services

- 1 Stripe naar productie**
Account activeren, KYC doorlopen, live keys instellen.
- 2 Supabase RLS configureren**
Row Level Security policies instellen in Supabase dashboard. Kritiek voor GDPR.
- 3 Identity verificatie service**
Onfido of Sumsub: account aanmaken, pricing kiezen, API keys verkrijgen.
- 4 Email service opzetten**
Provider kiezen (SendGrid, Resend, Postmark) en account aanmaken.
- 5 Domain & hosting**
Domein kopen, hosting kiezen (Vercel/Railway/Render), DNS instellen.
- 6 Push notification service**
Firebase Cloud Messaging of Expo Push Notifications account opzetten.

Juridisch & Compliance

- 7 Privacybeleid & Gebruiksvoorwaarden**
Juridisch verplicht (GDPR). Laat een jurist meekijken voor België.
- 8 GDPR compliance**
Data processing agreement, cookie beleid, recht op verwijdering.
- 9 Arbeidsrechtelijke check**
Platform-model juridisch laten checken. Studentenarbeid heeft specifieke regels in BE.
- 10 Btw-registratie**
Commissie innen = waarschijnlijk btw-plichtig. Check met boekhouder.
- 11 Contracttemplates nakijken**
Auto-gegenereerde contracten moeten juridisch waterdicht zijn.
- 12 Verzekering**
Arbeidsongevallen, aansprakelijkheid bij schade door student.

Design, Testen & Lancering

- 13 Figma designs finaliseren**
Definitieve designs met branding, kleuren en typografie goedkeuren.
- 14 Logo & branding**
Professioneel logo en huisstijl laten ontwerpen.
- 15 Beta testers werven**
Echte studenten en clients laten testen op hun eigen apparaten.

- 16** **Apple Developer Account**
\$99/jaar — nodig om iOS app te publiceren.
- 17** **Google Play Developer Account**
\$25 eenmalig — nodig voor Android publicatie.
- 18** **Payment flow testen**
Echte transactie doen en ontvangen, niet alleen test mode.
- 19** **Student verificatieproces**
Wie voert live meetings? Wat zijn de criteria? Operationeel proces.
- 20** **Klantenservice inrichten**
Klachten, geschillen, refunds — hoe ga je hiermee om?
- 21** **Marketing & acquisitie**
Strategie voor eerste studenten en clients.
- 22** **Monitoring tool kiezen**
Sentry of LogRocket voor foutmonitoring. Account is aan jou.

Prioriteiten Roadmap

FASE 1: BEVEILIG DE BASIS

- JWT middleware + RBAC (Claude)
- Input validatie (Claude)
- CORS + Rate limiting (Claude)
- Supabase RLS configureren (Denis)

FASE 2: CORE FEATURES AFMAKEN

- No-show beleid + job lock (Claude)
- Start-stop timer (Claude)
- Minimumfacturatie logica (Claude)
- Notificatiesysteem (Claude)
- Juridische check studentenarbeid (Denis)

FASE 3: GEBRUIKERSERVARING

- Error handling + loading states (Claude)
- Chat/messaging (Claude)
- Token-systeem (Claude)
- Component refactoring (Claude)
- Figma designs finaliseren (Denis)

FASE 4: PRODUCTIE-READY

- Email integratie (code) (Claude)
- Monitoring integratie (code) (Claude)
- API documentatie (Claude)
- Email service account (Denis)
- Stripe live mode (Denis)
- Hosting & domein (Denis)
- Monitoring account (Denis)

FASE 5: LANCERING

- Contractgeneratie (Claude)
- Database migraties (Claude)
- Beta testing (Denis)

● App Store accounts (Denis)

● Marketing (Denis)

● = Claude kan doen

● = Denis moet doen